

ABSTRAÇÃO DE COMPLEXIDADE: DESENVOLVIMENTO DE COMPILADORES PARA SISTEMAS DE AUTOMAÇÃO E ROBÓTICA INDUSTRIAL

Autores: Gabriel Fregolente - RA: 312410223

Marcio Borba Filho – RA: 312410086

Vinicius Silva Almeida – RA: 312411286

Ícaro Mendes Carbonez – RA: 312611719

Curso: Ciência da Computação - **Disciplina:** Teoria da computação e compiladores

Orientador: CLEBER SILVA FERREIRA DA LUZ

Data: 02/06/2026

Resumo

A automação industrial moderna depende fortemente de linguagens específicas de domínio (DSLs) capazes de descrever, com clareza, a rotina de robôs e o tratamento de sensores. Este trabalho apresenta o projeto e a implementação de um compilador para a linguagem **EasyBot**, uma DSL didática voltada à descrição de movimentos de robôs e ao controle baseado em sensores lógicos. O compilador foi desenvolvido em Java 17 com Maven e cobre as quatro fases clássicas — análise léxica, análise sintática, análise semântica e geração de código intermediário (**.mcode**). Um simulador de máquina de pilha foi implementado para executar o código gerado e validar a corretude do processo. Os resultados mostram que o compilador detecta corretamente erros léxicos, sintáticos e semânticos, em especial o uso indevido de variáveis do tipo **LOGICO** em expressões aritméticas, e gera código executável coerente com a especificação.

Palavras-chave: Compiladores. Linguagens de Domínio Específico. Automação Industrial. Análise Sintática. Robótica.

Resumo

Este trabalho apresenta o projeto e a implementação de um compilador para a linguagem **EasyBot**, uma linguagem específica de domínio voltada à descrição de movimentos robóticos e ao controle baseado em sensores lógicos. Desenvolvido em Java 17 com Maven, o compilador contempla as quatro fases clássicas da compilação: análise léxica, análise sintática, análise semântica e geração de código intermediário (**.mcode**). Além disso, foi implementado um simulador de máquina de pilha para executar o código gerado e validar a corretude do processo. Os resultados obtidos demonstram a capacidade do compilador de identificar corretamente erros léxicos, sintáticos e semânticos,

especialmente o uso inadequado de variáveis do tipo **LOGICO** em expressões aritméticas, bem como gerar código executável coerente com a especificação da linguagem.

1 Introdução

A Indústria 4.0 ampliou o uso de robôs colaborativos, células de manufatura flexível e sensores embarcados. Programar esses equipamentos diretamente em linguagens de uso geral, como C, Java e Python, exige conhecimento técnico amplo e produz código pouco legível para os profissionais de chão de fábrica. Uma alternativa consolidada é o uso de **linguagens de domínio específico** (DSLs), capazes de oferecer uma sintaxe mais próxima do vocabulário do processo industrial (AHO et al., 2008).

Este trabalho descreve o desenvolvimento de um compilador para a linguagem **EasyBot**, uma DSL didática proposta no edital da disciplina de Compiladores. A linguagem oferece comandos de movimento (**MOVER ... VEL ...**), declaração de variáveis tipadas (**NUMERO** e **LOGICO**), estruturas condicionais (**SE/SENAO**) e laços de repetição (**ENQUANTO**). O compilador, escrito em Java 17, executa as quatro fases clássicas e gera um código intermediário do tipo máquina de pilha (**.mcode**), executado por um simulador integrado.

O restante do artigo está organizado da seguinte forma: a Seção 2 revisa os fundamentos de compiladores empregados; a Seção 3 detalha a arquitetura do EasyBot; a Seção 4 apresenta a implementação; a Seção 5 discute os resultados; e a Seção 6 conclui o trabalho.

2 Fundamentação Teórica

2.1 Linguagens Formais e Autômatos

A análise léxica de uma linguagem regular é tradicionalmente implementada por um **autômato finito determinístico** (AFD), que reconhece padrões como identificadores, números e operadores (HOPCROFT; ULLMAN, 2002). Já a análise sintática de uma linguagem livre de contexto pode ser implementada por **parsers descendentes recursivos**, em que cada não terminal da gramática é representado por uma função (AHO et al., 2008).

2.2 Gramáticas EBNF

A *Extended Backus–Naur Form* (EBNF) é a notação padrão para a especificação formal de linguagens de programação. Ela permite descrever, de forma concisa, alternativas, repetições e opcionais, recursos indispensáveis ao se projetar uma DSL.

2.3 Análise Semântica e Tabela de Símbolos

A **tabela de símbolos** registra os identificadores declarados, seus tipos e escopos. Ela é consultada na análise semântica para detectar usos de variáveis não declaradas, incompatibilidades de tipo e outras violações que escapam à gramática (LOUDEN, 2004).

2.4 Geração de Código Intermediário

A geração de código intermediário desacopla o *front-end* do *back-end*. Representações comuns incluem código de três endereços e **código de máquina de pilha**. Esta última foi adotada no EasyBot por sua simplicidade de geração e por ser facilmente simulada.

3 A Linguagem EasyBot

3.1 Gramática EBNF

```
<programa> ::= "PROGRAMA" <id> "INICIO" <declaracoes> <bloco> "FIM"
<declaracoes> ::= ( "VAR" <id> ":" <tipo> ";" ) *
<tipo> ::= "NUMERO" | "LOGICO"
<bloco> ::= ( <comando> ) *
<comando> ::= <atribuicao> | <movimento> | <repeticao> | <condicional>
<atribuicao> ::= <id> "=" <expressao> ";"
<movimento> ::= "MOVER" "(" <expressao> "," <expressao> ")" "VEL" <expressao> ";"
<repeticao> ::= "ENQUANTO" <expressao_logica> "FACA" <bloco> "FIM_ENQUANTO"
<condicional> ::= "SE" <expressao_logica> "ENTAO" <bloco> ( "SENAO" <bloco> ) ?
"FIM_SE"
<expressao> ::= <termo> ( ( "+" | "-" ) <termo> ) *
<termo> ::= <fator> ( ( "*" | "/" ) <fator> ) *
<fator> ::= <numero> | <id> | "(" <expressao> ")" | "VERDADEIRO" | "FALSO"
<expressao_logica> ::= <expressao> ( "==" | "!=" | ">" | "<" ) <expressao>
<id> ::= [a-zA-Z][a-zA-Z0-9_]*
<numero> ::= [0-9]+ ( "." [0-9]+ ) ?
```

3.2 Palavras Reservadas

PROGRAMA, INICIO, FIM, VAR, NUMERO, LOGICO, MOVER, VEL, ENQUANTO, FACA, FIM_ENQUANTO, SE, ENTAO, SENAO, FIM_SE, VERDADEIRO, FALSO.

3.3 Tipos

A linguagem oferece dois tipos primitivos. **NUMERO** corresponde a valores reais de dupla precisão, suficientes para representar coordenadas e velocidades. **LOGICO** representa estados booleanos provenientes de sensores. Uma das regras semânticas centrais proíbe o uso de variáveis **LOGICO** em expressões aritméticas, a fim de evitar comportamentos indefinidos em tempo de execução.

4 Implementação

4.1 Arquitetura

O compilador é organizado em pacotes Java independentes, refletindo as quatro fases do processo de compilação: **lexer** (Scanner, Token, TipoToken), **parser** (Parser descendente recursivo), **ast** (nós da AST com padrão Visitor), **semantico** (TabelaSimbolos e AnalisadorSemantico), **codegen** (GeradorMCode) e **runtime** (SimuladorMCode). A classe **Main** integra as fases e implementa a interface de linha de comando.

4.2 Análise Léxica

O scanner foi implementado a partir de um autômato finito determinístico simplificado. Espaços em branco e comentários (*//* e */* */*) são descartados; identificadores são comparados a uma tabela de palavras reservadas; e números podem conter parte fracionária. De forma resumida, letras e sublinhado iniciam o reconhecimento de identificadores e palavras reservadas, enquanto dígitos iniciam o reconhecimento de números inteiros e reais. Operadores relacionais como **==** e **!=** são tratados com estados específicos, e os demais símbolos, como **+**, **-**, *****, **/**, **;**, **:**, **,**, **>** e **<**, são aceitos diretamente.

Cada **Token** carrega tipo, lexema, linha e coluna, permitindo mensagens de erro precisas nas fases posteriores.

4.3 Análise Sintática

O parser é descendente recursivo e segue diretamente a EBNF: cada não terminal se transforma em um método, como **parsePrograma**, **parseBloco** e **parseComando**. A construção é incremental e resulta em uma **Árvore Sintática Abstrata** (AST) cujos nós implementam o padrão *Visitor*, simplificando as fases posteriores.

4.4 Análise Semântica

A **TabelaSímbolos** armazena, para cada identificador declarado, nome, tipo e linha. O **AnalizadorSemantico** percorre a AST acumulando erros em uma lista e, ao final, lança uma única exceção contendo todas as inconsistências detectadas.

- declaração obrigatória antes do uso;
- compatibilidade de tipo em atribuições;
- operandos de **+**, **-**, ***** e **/** devem ser **NUMERO**;
- coordenadas e velocidade do comando **MOVER** devem ser **NUMERO**;
- condições de **SE** e **ENQUANTO** devem ser **LOGICO**;
- comparações exigem tipos compatíveis, com operandos numéricos para **>** e **<**.

4.5 Geração de Código

O **GeradorMCode** traduz a AST para um código intermediário de **máquina de pilha**, escrito em arquivo **.mcode**. As instruções utilizadas são apresentadas no Quadro 1.

Quadro 1 — Conjunto de Instruções da Máquina Virtual EasyBot

Instrução	Efeito
PROGRAM nome	Cabeçalho do programa
DECL x TIPO	Declara variável x do tipo TIPO
PUSHN v	Empilha número v
PUSHB true/false	Empilha booleano
LOAD x	Empilha o valor de x
STORE x	Desempilha o topo e grava em x

ADD/SUB/MUL/DIV	Operações aritméticas
EQ/NEQ/GT/LT	Operações relacionais
LABEL n	Marca rótulo
JZ n	Salta para n se o topo for falso ou zero
JMP n	Salto incondicional
MOVE	Consome vel, y, x e executa movimento
HALT	Encerra a execução

4.6 Simulador

O **SimuladorMCode** interpreta as instruções geradas e mantém uma pilha de execução e um mapa de variáveis. Cada comando **MOVE** produz uma linha de log descrevendo o destino e a velocidade, simulando o comportamento do robô.

5 Resultados

Foram testados três programas-exemplo, conforme descrito a seguir.

5.1 patrulha.eb

O programa descreve três voltas em uma trajetória retangular. A saída do simulador exibe os doze movimentos esperados, quatro por volta, confirmando o funcionamento da instrução de repetição e do gerador de código.

```
[INIT] programa Patrulha
[MOVE] destino=(10,00, 0,00) vel=2,00
[MOVE] destino=(10,00, 10,00) vel=2,00
[MOVE] destino=(0,00, 10,00) vel=2,00
[MOVE] destino=(0,00, 0,00) vel=2,00
... (repetido 3 vezes)
[HALT]
```

5.2 obstaculo.eb

Esse exemplo demonstra a reação a um sensor lógico (**sensor_obstaculo**), combinando estruturas **SE/SENAO** e **ENQUANTO**. O compilador aceita o programa sem erros e o simulador executa o trajeto previsto.

5.3 erros.eb

O programa foi elaborado propositalmente com inconsistências, a fim de validar a análise semântica. A execução produziu a lista exata de erros esperada.

Foram encontrados 4 erros semânticos:

- linha 9: operando esquerdo de '+' deve ser NUMERO (achou LOGICO)
- linha 10: variável 'distancia' não declarada

- linha 12: condição de SE deve ser LOGICO (achou NUMERO)
- linha 13: coordenada X de MOVER deve ser NUMERO (achou LOGICO)

Os resultados demonstram que todas as fases do compilador estão operacionais e que as regras semânticas exigidas pelo edital são aplicadas corretamente.

6 Considerações Finais

O trabalho atingiu o objetivo de projetar e implementar um compilador funcional para uma DSL voltada à automação industrial. A escolha por uma arquitetura modular, contemplando análise léxica, sintática, semântica, geração e simulação, facilitou tanto o desenvolvimento quanto a validação. A geração de código intermediário em forma de máquina de pilha mostrou-se adequada como abstração executável.

Como trabalhos futuros, propõe-se: (i) suporte a procedimentos e parâmetros; (ii) integração com bibliotecas de simulação robótica reais, como ROS ou *PyBullet*; (iii) geração de código para controladores lógicos programáveis (CLPs); e (iv) inclusão de testes automatizados com JUnit cobrindo cada fase do compilador.

Referências

AHO, Alfred V. et al. **Compiladores: princípios, técnicas e ferramentas**. 2. ed. São Paulo: Pearson, 2008.

HOPCROFT, John E.; ULLMAN, Jeffrey D. **Introdução à teoria de autômatos, linguagens e computação**. 2. ed. Rio de Janeiro: Campus, 2002.

LOUDEN, Kenneth C. **Compiladores: princípios e práticas**. São Paulo: Cengage Learning, 2004.

PRICE, Ana M. A.; TOSCANI, Simão S. **Implementação de linguagens de programação: compiladores**. 4. ed. Porto Alegre: Bookman, 2008.

SEBESTA, Robert W. **Conceitos de linguagens de programação**. 11. ed. Porto Alegre: Bookman, 2018.